



Evaluating your ML project with the MLOps checklist

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Evaluating your ML project with the MLOps checklist

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction

Attachments

MLOps checklist components

1. Data-centric management

2. Experimentation

3. Observability and model management

4. Robust pipelines and promotion

5. Continuous integration

6. Continuous monitoring

7. Continuous deployment

8. Continuous training

9. Governance

Example usage

Resources

References

Additional reading

Tools

Consulting

Document history

1

1

2

2

4

6

8

10

13

16

18

19

21

22

22

22

22

22

23

Evaluating your ML project with the MLOps checklist

Charles Frenzel, Sharath Nagaraja, and Spencer Romo, Amazon Web Services

The MLOps checklist is a workable checklist that you can use at any phase in your machine learning (ML) project. The checklist is a tool for assessing overall readiness, examining system coverage, and identifying new areas of opportunity in distributed ML systems. MLOps is the combination of people, technology, and processes for delivering ML solutions. Well-architected MLOps helps businesses to deploy ML models to production effectively and consistently, and can deliver business value.

Using the MLOps checklist helps you to do the following:

- Assess your MLOps system.
- Find areas of opportunity.
- Find areas for improvement.
- Evaluate and update your strategic roadmap on AWS.
- Generate backlog items.

We recommend using the MLOps checklist at the start of your MLOps project, but it's possible to use parts of it during any phase.

Attachments

To access additional content that is associated with this document, download and unzip the following file:

[attachment.zip](#)

MLOps checklist components

The MLOps checklist consists of nine different components. Each component represents an aspect of a mature MLOps system. This section covers the checklist components in detail. *Note that there is some overlap in this section because certain components depend on other components.*

The checklist covers the following areas:

- Data-centric management
- Experimentation
- Observability and model management
- Robust pipelines and promotion
- Continuous integration (CI)
- Continuous monitoring
- Continuous deployment (CD)
- Continuous training
- Governance

Note

Security is already well-covered by AWS. We recommend looking at security-related AWS Prescriptive Guidance [guides](#) and [patterns](#), and reviewing [AWS security best practices](#).

1. Data-centric management

Data management is the practice of ensuring that data used in training, testing, and inference is properly managed, secured, and validated. When building models at scale, data is the primary commodity that enables high model performance.

1.1 Data repository

A data repository requires the ability to track data and see its point of origination. When new data is added or removed, the data repository records those changes in point-

in-time recovery. The data repository should take into account how label data is tracked and processed, and how intermediate data artifacts are tracked.

1.2 Diverse data source integration

Depending on the application, training your model might require data from many sources. Designing and maintaining a manifest that informs ML practitioners of the available data sources and how they tie together is critical to building models.

1.3 Data schema validation

To feed models data, it's important that the training data be homogeneous. Transformations or other exploratory analysis might be required for data that is stored in data lake solutions such as Amazon Simple Storage Service (Amazon S3) or in document data stores.

1.4 Data versioning and lineage

When training models that might be used in production, you must be able to reproduce results and have a reliable way to perform [ablation studies](#) to better understand the overall model performance. Tracking the state of the training data is critical to this reproducibility. Tools such as [Data Version Control \(DVC\)](#) can assist with this.

1.5 Labeling workflow

In cases where labeled data is not available at the project start, creating labeled data is often a necessary step. Tools such as Amazon SageMaker Ground Truth require input data to be appropriately structured, and they require a defined and tested labeling job. A workforce of either internal or external labelers must be used. Data should then be validated, using either redundant labeling or machine learning approaches to identify outliers or errors in the training dataset.

1.6 Online and offline feature storage

The ML system has a [Feature Store](#) or a centralized store for features and associated metadata so that it's possible to reuse features, or model inputs. You can create an online or an offline store. Use an online store for low-latency, real-time inference use cases. Use an offline store for training and batch inference.

2. Experimentation

Experimentation covers experiment logging, tracking, and metrics. This translates to experiment metadata integration across the platform, in source control, and in development environments. Experimentation also includes being able to optimize model performance and accuracy through debugging.

2.1 Integrated development environments

An integrated development environment (IDE) is integrated directly with the cloud. The IDE can interact with and submit commands to the larger system. Ideally, it supports the following:

- Local development
- Version control integration

- Debugging in place, with all logs and artifacts generated going into the version control

2.2 Code version control

To help ensure reproducibility and reusability, all code is committed into the source repository with proper version control. This includes infrastructure code, application code, model code, and even notebooks (if you opt to use them).

2.3 Tracking

An ML project requires a tool that can track and analyze machine learning experiments. This tool should log all metrics, parameters, and artifacts during a machine learning experiment run, recording all metadata into a central location. The central location will provide the ability to analyze, visualize, and audit all experiments that you run.

2.4 Cross-platform integration

Historical results for experiments and all their metadata are accessible in other parts of the system. For example, the orchestration pipelines in place can access this data, as can the monitoring tools.

2.5 Debugging: accuracy and system performance

A comprehensive model debugging framework is in place to examine runs for the following:

- Find bottlenecks
- Alert about anomalies
- Maximize resource utilization
- Aid in analysis of experiments

When training is intensive, the ability to maximize throughput is crucial and makes this a necessary tool for cost optimization.

3. Observability and model management

The observability and model management section of the checklist encompasses model version control and lineage tracking across the entire ML system. Model versioning helps to track and control all changes applied to a model so that you can recover a previous version when needed. Lineage tracking provides a view into model inflows and outflows. Another key benefit of lineage tracking is point-in-time recovery (PITR), which automates deployment and system recovery.

3.1 Versioned model registry

In general, a model registry supports version control and lineage tracking of model components. A good registry can associate metadata with the versioned model, including the following:

- The data used
- Information about the model
- Evaluation metric results
- Associated model code

3.2 Bias, fairness, and explainability

At a bare minimum, an ML system should have a process whereby a model's predictions are explainable to other parties. Users should be able to check results for bias by each feature. Ideally, measure data bias before inputting the data into the ML model, and record these metrics for model cards and auditing.

3.3 Lineage tracking: data inputs and outputs

Tracking is in place to follow the flow of data in and out of the system (for example, runs from the data lake to the training pipeline). This tracking acts as a record from which all system processes can be recreated, and it provides an audit trail for analysis.

3.4 Lineage tracking: environment information

This tracking captures information about the runtime environment setup, such as container

images for all model code and the containers' associated dependencies.

3.5 Lineage tracking: model

This tracking captures information about the model. It includes everything from information on the model's algorithm to parameters and hyperparameters that go into the model.

3.6 Integration with deployment and monitoring

The system should be linked directly with monitoring and deployment subsystems for PITR. For monitoring, this means testing the model's performance against its training runs to detect model-quality deterioration. For deployment, this supports PITR and the ability to roll back to a previous model version as needed.

3.7 Pipeline parameter configuration

Technically, pipeline parameter configuration falls under both lineage tracking and experiment tracking because the pipeline configuration must be versioned and associated directly with a model. Pipeline parameter configuration is listed in this section because it's imperative to track all system orchestration configurations and version them.

3.8 Issues are traceable, debuggable, and reproducible.

An engineer can trace, debug, and reproduce all issues within the system without much effort. This implies that a sufficient level of observability is in place. This check is primarily derived from fulfilling the other items under the Observability and model management section.

3.9 Performance visualization

The system can capture and gather logs into a time-series database type format and ingest them directly into dashboard. The dashboard provides a holistic view of both model and computer metrics with the ability to drill down and query.

4. Robust pipelines and promotion

Pipelines provide many options for hyperparameter tuning, AutoML, and processing routines. Pipelines are logged from end to end. Robust pipelines can run training in parallel across multiple instances and frameworks, scaling load sizes as needed. Robust pipelines can promote models into production, deploying in real-time, streaming, and batch. These deployments can support single-model or multi-model inference.

4.1 Large-scale and distributed training

A mature ML system supports the ability to run training on large compute-optimized instances in parallel. It has the tooling in place to help ensure that these resources are fully used and that the training scales evenly across the compute cluster.

4.2 Support for multiple frameworks

Developers can port different platform frameworks, such as PyTorch or Flax, to run training and inference jobs. Likewise, different languages and versions are supported and usable. Switching to another framework will not break the system.

4.3 Hyperparameter tuning

A hyperparameter tuning step is a part of the training pipeline. Deployed models have their hyperparameters tuned. Multiple options are available to tune hyperparameters. For accuracy improvement, at least one of the tuning options should have a Bayesian inference or approach.

4.4 AutoML option

To reduce manual experimentation and comparison, a mature ML system supports running AutoML, which automatically selects the best feature pipeline, hyperparameters, and model. Note that AutoML is a feature to use pragmatically, but it's not a panacea.

4.5 Inference support: real time

This is commonly called model as a service (MaaS). The system supports real-time inference with REST API operations, for inference request on demand. It's able to ship MaaS infrastructure on which the model can scale both horizontally and vertically as a standalone API or as an endpoint associated with other applications. Alternatively, it's possible to deploy by using serverless technology.

4.6 Inference support: streaming

Models can be promoted to a real-time inference format such as Amazon Kinesis or Amazon Managed Streaming for Apache Kafka, whereby inference is run in streaming fashion on the model. This requires at least 90 percent of the checklist to be complete, because guardrails, observability, and monitoring are essential for real-time inference.

4.7 Inference support: batch

The system supports batch deployment of models as scheduled or initiated jobs. The system can run models as part of an extract, transform, and load (ETL) process or in isolation. Batch jobs record the state from each step and run in an ordered pattern, such as a directed acyclic graph. Alternatively, jobs can write to a database, which acts as the server of model inference.

4.8 Preprocessing and post-processing routines

When needed, data is featurized as part of the model intake process or the batch jobs. If there are multiple models or multiple steps at play, post-processing routines take care of featurizing the data.

4.9 Ability to invoke hierarchical or simultaneous models

The ML system can deploy many models together or run them in a sequential fashion. The former means hosting on a single model endpoint across a fleet of resources. The latter implies that multiple models need to run in a chained fashion one after the other. The system can handle both these types of complexity resiliently.

4.10 Horizontal and vertical scaling strategies

A pipeline should have the ability to support both types of scaling strategies for training and inference. The ML system can increase its size and distribute traffic across multiple machines when either latency or throughput increases. Policies for this type of behavior are set, and they consider optimal resource allocation.

4.11 End-to-end logging

The development team should have logging set inside all pipeline code so that logging can capture inputs, outputs, and intermediate steps in the system. Logging should support tracing runs in the pipeline and debugging errors.

5. Continuous integration

The ML system runs tests to validate that the system works from end to end, checking for possible points of failure. Tests are run automatically on commit, and longer tests are run on a fixed

schedule. Tests check traditional software engineering areas such as at the unit and system level. In addition, tests capture the particulars of ML by checking data, features, and the model.

5.1 Local code checks

Before committing code into a centralized code repository, developers locally run checks such as basic unit tests and static analysis. Running these checks before committing increases overall code quality and catches problems before they enter version control.

5.2 Static code analysis

The central code repository has static code analysis tools that run on commit, quickly. This tooling should improve code style and formatting. It should also check for common security vulnerabilities within source and infrastructure code, common bugs, and other weaknesses in the code.

5.3 Data quality tests

Data quality tests should, at a bare minimum, check that the data has not violated a fixed schema. A more comprehensive approach is to compute data statistics at ingest, set constraints on the data, and run tests against these.

It's possible to set up data quality tests independently or as part of the pipeline. The statistics and constraints are reused for monitoring.

5.4 Feature tests

As part of a complete pipeline, feature importance is generated. Feature tests assert that the importance of features, or the model's way of attributing feature values, does not change. Feature tests can feed into monitoring because they can alert and track violations in a model's inputs.

5.5 Unit tests

Unit tests for all code—model, application, and infrastructure—run before commit and on commit. Each unit test provides a check on an important piece of code to confirm that it functions as expected. In the case of ML code, tests can run for algorithmic correctness.

5.6 Integration tests

An integration test verifies that the pipeline runs end to end successfully, including standing up the associated infrastructure for the pipeline. This test validates that the system is working and logging as expected. If deployment is separate, there should be an end-to-end test for this as well to make sure deployment works.

5.7 Smoke tests

The system has smoke tests that run in mini and rapid regression of each piece of functionality. Smoke tests are part of continuous integration, and can run in a containerized environment to mimic cloud functionality.

5.8 Load testing

On-demand load testing is in place. In addition to capturing how the ML system behaves under high and low loads, load tests provide statistics on system-wide throughput or latency. Data gathered through load tests provides information about resource sizes and scaling policies.

5.9 Model functional tests

Model outputs and inputs run through automated functional tests. To check a behavior within a capability, both outputs and inputs for the model are tested on real or fake data with basic examples.

5.10 Model inference tests with extreme cases

As part of the minimum functionality testing, model tests should check for extreme behavior given certain inputs before model promotion . This places an additional guardrail to help prevent unexpected behavior.

6. Continuous monitoring

In continuous monitoring, automated processes observe and detect performance issues and model issues. Owners can then identify potential problems and threats in real time to address them quickly.

Continuous monitoring surfaces possible model issues such as data quality, distribution shift, model concept shift, and model quality degradation. Continuous monitoring also includes comprehensive logging for traditional system measures such as saturation, latency, traffic, and errors. A practical notification and alert strategy are set up to notify owners when issues arise.

6.1 Model monitoring: data quality detection

Rule-based monitoring is in place to know when incoming data deviates from model training data. This type of monitoring creates a schema from the training data, sets constraints based on that schema, and then runs exceptions when a violation occurs.

6.2 Model monitoring: distribution shift

Monitoring is set up to look at the incoming data distribution and check that it hasn't deviated from the model training data distribution. For example, the incoming data is sampled in [as a moving window](#) over inference data. A job is then run to test the sampled distribution and training distribution to see if they are the same.

6.3 Model monitoring: model concept drift

A concept drift check looks for the relationship between a model's inputs and target variable to remain unchanged from the training data.

An additional check is to confirm that the relative features and their importance don't change.

6.4 Model monitoring: model evaluation check

This is a monitoring check that evaluates whether the model's quality has degraded. The model evaluation check compares baseline evaluation metrics from training time with the incoming results to assess whether the model's accuracy level has decreased on new data. Because it computes accuracy metrics, this check requires the ground truth of new data to be available after inference.

6.5 System captures: input schemas

The ML system captures the schema of training, testing, and validation data. In addition to providing information about inputs, schemas provide statistics regarding their skew and completeness. Schemas are used for immediate testing and data quality monitoring checks in production.

6.6 System captures: evaluation results and statistics

The ML system outputs accuracy information on validation and training data. It can output the predictions and true labels from validation and training runs. These are used as monitoring constraints for the live production model.

6.7 System captures: anomalies

There is a tracking mechanism in place to flag anomalies in incoming data streams. If outliers occur in incoming data or if during a specified timeframe the key feature distribution changes, the system recognizes this as an anomaly and flags it.

6.8 Logging: saturation and resources

There is logging in place for how full the system is. Resource and saturation metrics should focus on CPU utilization, graphics processing unit (GPU) utilization, memory utilization, and disk utilization. These metrics should be available in time-series format with the ability to measure in percentiles. For batch jobs, this provides information on throughput, which shows how many units of information the system can process in each amount of time.

6.9 Logging: latency

Logging should be in place to measure the delay in network communication or the time it takes to service a request. An engineer should be able to judge how long the inference models are taking to serve predictions and how long the model takes to load.

6.10 Logging: traffic

The logging setup for traffic measures the volume of traffic on each instance. Traffic is measured by the number of HTTP requests and bytes or packets sent or received during a certain amount of time. Logging traffic provides insights of the total workload that is placed on a system.

6.11 Logging: errors

The logging setup for errors captures the number of requests that fail. Failures are of the following types:

- Explicit (for example, HTTP 500 errors)
- Implicit (for example, an HTTP 200 success response that's coupled with the wrong content)
- Policy (for example, if you commit to one-second response times, any request over one second is an error)

Where protocol response codes are insufficient to express all failure conditions, secondary (internal) protocols might be necessary to track partial failure modes.

6.12 Notifications and alerting

Notifications and alerts are set up from monitoring. Notifications include the ability to get Slack, email notification, pages, and Short Message Service (SMS) messages. Alerting doesn't mean sending notifications for all possible violations. Instead, it means setting alerts to specific exceptions that are meaningful and important to the development team. In this way, alert fatigue is avoided.

7. Continuous deployment

For an ML system to be continuously deployed, it must be able to divert traffic from or between live models. A continuously deployed system has at least one way by which models are promoted to production: canary, shadow, blue/green, or A/B. Confirm that in the ML system, you have at least one way to roll back models.

7.1 Model switching

The system can switch between versioned models in staging and production. It can divert traffic all at once or incrementally to new production variants.

7.2 Model promotion processes

A staged validation process is in place for model promotion. The process uses offline tests that don't impact the production system, such as running against validation data in a staging environment. A runbook and metrics for model promotion are set. Promotion follows one of the rollout strategies.

7.3 Rollback strategies

A rollback strategy exists so that when an error occurs or the model deviates from expected behavior, a rollback, fallback, or roll through happens. In a rollback, the model reverts to a previous deployment version. In a fallback, the model is replaced with a strong heuristic. Roll through will promote the next model to production, rolling through the previous model. Runbooks are in place for all of these strategies.

7.4 Canary deployment

The system can deploy by using a canary. A small portion of traffic is sent to the new model initially. Over time, all traffic shifts to the new model. This shift is closely monitored because the testing happens in the production environment.

7.5 Model shadow deployment

The system can run a shadow deployment in which the new model works alongside the existing model. Both models receive traffic, but only the earlier model outputs inference. Assessments are run on the new model

compared with the existing model, and then the new model is manually promoted.

7.6 Blue/green deployment

The system can deploy with a new model (green, which is staging) and the earlier model (blue, which is production), with both running at the same time. After testing is complete, traffic is diverted from the blue environment to the green one. This strategy prevents downtime because identical environments are stood up.

7.7 Support for A/B testing or more

The system supports using model versions in the deployed environment to run A/B tests on incoming traffic. This can include the ability to promote automatically based on the newer model winning in the tests. More advanced setups will use a [multi-armed bandit](#) process.

8. Continuous training

Continuous training means that the ML system automatically and continuously retrains machine learning models to adapt to changes in the data before it is redeployed. Possible triggers for rebuilding include data changes, model changes, or code changes.

8.1 Checks: model input validation

Checks are in place to verify a model's input doesn't deviate from a certain standard. Input validation means running functional testing during model promotion. It also means having immediate verification of input requests, such as using assertions and enumerated types.

8.2 Retrain triggering: scheduled jobs

This is the most basic form of training automation. Model retraining is set on a schedule (for example, every week). In this scenario, automation is likely low, with a

manual review and spot check on the results before model promotion.

8.3 Retrain triggering: new training data

Retraining is initiated by an incoming data threshold. The model can retrain from scratch or run updates incrementally. Given a specified amount of data in place, a training job kicks off.

8.4 Retrain triggering: model performance degradation

This technique uses monitoring and observability to run model retraining, and it requires a mature level of automation. For example, accuracy lowers from a given range, which acts as a trigger for retraining a model on all or part of the data.

8.5 Retrain triggering: data distribution shift

Monitoring data distribution shift provides a way set triggers to retrain the model when its underlying data changes. A violation set on [concept shift or data distribution shift](#) initiates a model retraining job.

9. Governance

ML governance encompasses a set of processes and frameworks that help in the deployment of ML models. It includes model explainability, auditability, traceability, and other more abstract but essential requirements of a successful end-to-end ML lifecycle.

9.1 Data quality and compliance

The ML system accounts for personal identifying information (PII) considerations, including anonymization. It has documented and reviewed column-level lineage for understanding the source, quality, and appropriateness of the data. It also has automated data quality checks for anomalies.

9.2 Audit and documentation

The ML system has a full log of all changes during development, including experiments run and reasons for choices made for regulatory compliance.

9.3 Reproducibility and traceability

The ML system includes a full data snapshot for precise and rapid model re-instantiation, or it has the ability to recreate the environment and retrain with a data sample.

9.4 Human-in-the-loop signoff

The ML system has manual verification and authorization for regulatory compliance. The system requires signoffs for every environment move (for example, Dev, QA, pre-Prod, and Prod).

9.5 Bias and adversarial attacks testing

The ML system has *Red Team* adversarial testing using multiple tools and attack vectors, and automated bias checking on specific subpopulations. This component ties back to the Observability and model management section.

Example usage

There is no one way to use the MLOps checklist (attached). A common pattern, however, is to iterate on it until system quality and performance are at the levels that you want.

An iteration cycle can look like the following:

1. Go through the checklist item by item.
2. Review the existing checklist results.
3. Identify any gaps.
4. Prioritize gaps for remediation.
5. Develop implementations for new features that you want.
6. Repeat the iteration cycle.

Resources

References

- David Nigenda, Zohar Karnin, Muhammad Bilal Zafar, Raghu Ramesha, Alan Tan, Michele Donini, and Krishnaram Kenthapadi. 2022. "Amazon SageMaker Model Monitor: A System for Real-Time Insights into Deployed Machine Learning Models": [arXiv:2111.13657](https://arxiv.org/abs/2111.13657).
- Richard Meyes, Melanie Lu, Constantin Waubert de Puiseau, and Tobias Meisen. 2019. "Ablation Studies in Artificial Neural Networks": [arXiv:1901.08644](https://arxiv.org/abs/1901.08644).
- Brian Babcock, Mayur Datar, and Rajeev Motwani. "Sampling From a Moving Window Over Streaming Data": [Princeton University](https://princetonuniversity.edu/).
- Chip Huyen. [*Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*](#). Sebastopol, California: O'Reilly Media, 2022.
- Cathy Chen, Niall Murphy, Kranti Parisa, D. Sculley, and Todd Underwood. [*Reliable Machine Learning: Applying SRE Principles to ML in Production*](#). Sebastopol, California: O'Reilly Media, 2022.

Additional reading

- [AWS Well-Architected Framework: Machine Learning Lens](#)
- [Detect NLP data drift using custom Amazon SageMaker Model Monitor](#) (blog post)
- [Dynamic A/B testing for machine learning models with Amazon SageMaker MLOps projects](#) (blog post)

Tools

- [Amazon SageMaker](#)
- [Data Version Control \(DVC\)](#)

Consulting

- [AWS Professional Services](#)

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	July 26, 2023